Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

*Факультет программной инженерии и компьютерной техники*
*Направление подготовки: 09.03.04 – Программная инженерия, Системное и прикладное программное обеспечение*

Дисциплина: Информационные системы

# Лабораторная работа №2

Выполнил:
Карнажицкий Максим Романович
Группа: P3311

Проверил:
Тюрин И. Н.

г. Санкт-Петербург, 2026

# Содержание

1. Задание ..... 2
2. Диаграмма классов разработанного приложения ..... 4
3. Исходный код ..... 6
   3.1. Ссылка на репозиторий ..... 6
   3.2. Комментарии к исходному коду ..... 6
      3.2.1. Транзакции ..... 6
      3.2.2. Импорт файлов ..... 6
      3.2.3. Apache JMeter ..... 7
         3.2.3.1. Happy-path для создания фильма и импорта файла. ..... 7
         3.2.3.2. Concurrent race ..... 9
4. Вывод ..... 12

# 1. Задание

Доработать ИС из ЛР1 следующим образом:

- Добавить в систему возможность массового добавления объектов при помощи импорта файла. Формат для импорта необходимо согласовать с преподавателем. Импортируемый файл должен загружаться на сервер через интерфейс разработанного веб-приложения.
  - При реализации логики импорта объектов необходимо реализовать транзакцию таким образом, чтобы в случае возникновения ошибок при импорте, не был создан ни один объект.
  - При импорте должна быть реализована проверка пользовательского ввода в соответствии с ограничениями предметной области из ЛР1.
  - При наличии вложенных объектов в основной объект из ЛР1 необходимо задавать значения полей вложенных объектов в той же записи, что и основной объект.
- Необходимо добавить в систему интерфейс для отображения истории импорта (обычный пользователь видит только операции импорта, запущенные им, администратор - все операции).
  - В истории должны отображаться id операции, статус ее завершения, пользователь, который ее запустил, число добавленных объектов в операции (только для успешно завершенных).
- Согласовать с преподавателем и добавить в модель из первой лабораторной новые ограничения уникальности, проверяемые на программном уровне (эти новые ограничения должны быть реализованы в рамках бизнес-логики приложения и не должны быть отображены/реализованы в БД).
- Реализовать сценарий с использованием Apache JMeter, имитирующий одновременную работу нескольких пользователей с ИС, и проверить корректность изоляции транзакций, используемых в ЛР. По итогам исследования поведения системы при ее одновременном использовании несколькими пользователями изменить уровень изоляции транзакций там, где это требуется. Обосновать изменения.
  - Реализованный сценарий должен покрывать создание, редактирование, удаление и импорт объектов.
  - Реализованный сценарий должен проверять корректность поведения системы при попытке нескольких пользователей обновить иили удалить один и тот же объект (например, двух администраторов).
  - Реализованный сценарий должен проверять корректность соблюдения системой ограничений уникальности предметной области при одновре-

менной попытке нескольких пользователей создать объект с одним и тем же уникальным значением.

## 2. Диаграмма классов разработанного приложения

Диаграмма была создана мной вручную. На середине процесса я **очень** утомился переписывать сигнатуру методов репозиториев, поэтому воспользовался современными средствами (LLM). Так дело пошло в гору

Промпт:

> Тебе нужно составить список сигнатур функций для UML-диаграммы в таком формате: `+ funcName(arg1: Arg1Type, arg2: Arg2Type): ReturnType`. Верни только моноширинный список сигнатур функций класса, который я приложу далее. И еще его атрибуты в формате `+/- attrName: AttrType`

Для указания `DI` я использовал связь *ассоциации*. Связь обобщения использовалась, как и обычно, для указания наследования.

`DTO` в схеме не указаны - опущены, чтобы не перегружать схему.

Ссылка на диаграмму (она большая): https://github.com/xtern0o/is-lab2/blob/dev/is_lab2/docs/uml-is-lab2.drawio.svg

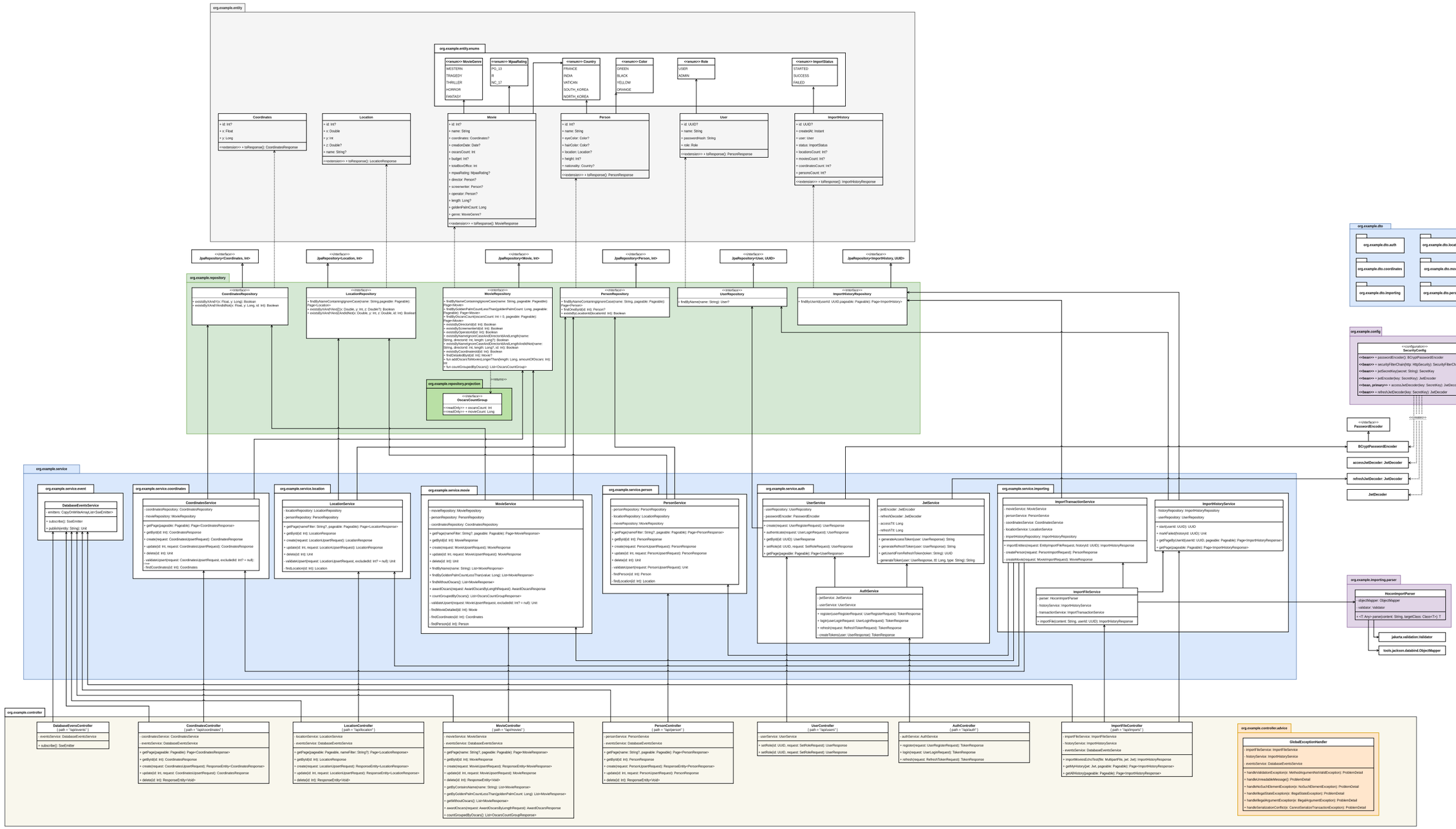



Рис. 1. UML-диаграмма классов

# 3. Исходный код

## 3.1. Ссылка на репозиторий

https://github.com/xtern0o/is-lab2

## 3.2. Комментарии к исходному коду

### 3.2.1. Транзакции

На все операции чтения на уровне `@Service` были навешены `@Transactional(readOnly = true)` для того, чтобы оптимизировать их на уровне `JDBC`, и чтобы Hibernate не делал dirty-checking (проверку изменений у загруженных сущностей) при завершении транзакции

Операции `delete` реализуют стандартный уровень изоляции по умолчанию (`ReadCommited`) - этого достаточно. В идеале хотел добавить `OptimisticLock` с версионированием сущностей, чтобы не удалять/редактировать несуществующие сущности в состоянии raceCondition, но решил пока не добавлять.

Операции `create/update` работают немного иначе. Из-за того, что есть бизнес-проверка уникальности сущностей, нужно полностью изолировать параллельные транзакции друг от друга, чтобы не было такого, что мы проверили уникальность, а перед добавлением новой сущности добавилась идентичная с точки зрения бизнес-требований, пока добавляется наша. В таком случае в БД будет 2 одинаковые сущности. Поэтому я выбрал уровень изоляции `SERIALIZABLE`, чтобы полностью исключить параллельность выполнения этих транзакций. Да, это ударит по производительности, однако корректность данных не будет нарушена.

### 3.2.2. Импорт файлов

По совету я выбрал формат данных `HOCON` (Human oriented configuration object notation) - он более гибкий, чем какой-нибудь json/yml/xml. Для него пришлось написать отдельный класс `HoconImportParser`, однако все, что н делает - использует библиотеку для парсинга, и если что - выбрасывает ошибку.

Для того, чтобы исключить конфликты записи, импорт файла - это одна `SERIALIZABLE` транзакция. Так, если хоть одна сущность окажется некорректной, или уже будет находяться в бд, или будут нарушены бизнес-требования - откатится вся транзакция и ни один объект не будет добавлен.

История импорта хранит количество **верхнеуровнево-созданных** сущностей. То есть вложенные сущности в расчет не идут. Я посчитал, что это проще реализовать, да и требований, как считать - нет.

Также история хранит пользователя, который отправил файл. В любом случае (успех/неудача) история сохраняется и любой пользователь получает доступ к просмотру своих посылок. Админ имеет возможность смотреть все.

Также добавил небольшую админ-панель, где админ может менять роли пользователей, чтобы не приходилось постоянно ходить в бд и писать `UPDATE users SET role='ADMIN' WHERE name='smth'`.

### 3.2.3. Apache JMeter

Для тестирования я выделил 2 ключевых ветки:

#### 3.2.3.1. Happy-path для создания фильма и импорта файла.

Неконкурентный флоу для стандартного поведения программы. unhappy-path (например, невалидные данные по бизнес-логике) можно проверить вручную.

На картинке я приложил скрин действий, более подробно - jmx файл на гитхабе. в `is_lab2/jmeter/TestPlan.jmx`.

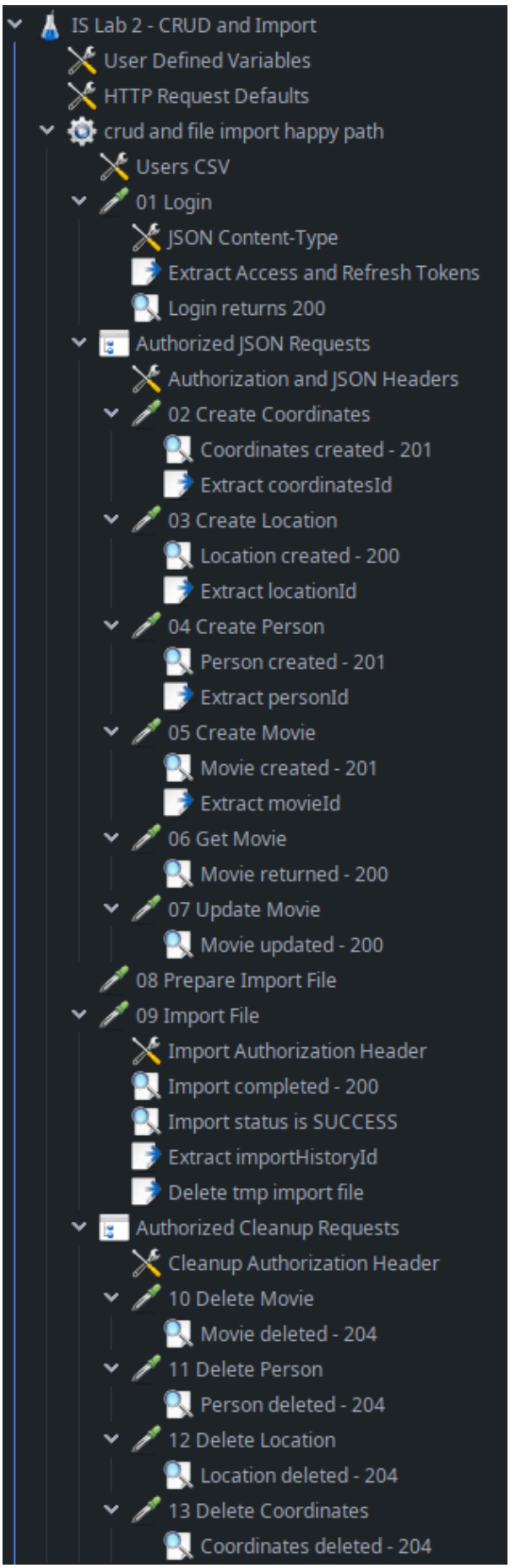



Рис. 2. Флоу действий для happy path

Тестирование прошло успешно - сущности созданы без ошибок.

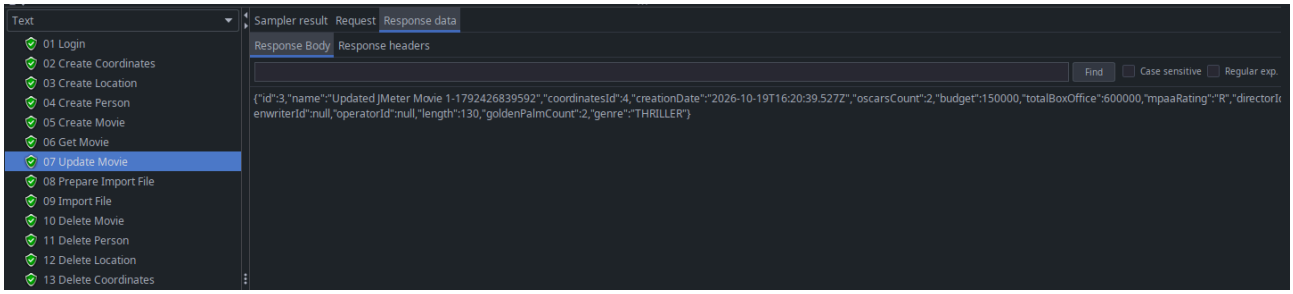



Рис. 3. Скрин успешного тестирования из View Results Tree

#### 3.2.3.2. Concurrent race

Второй сценарий я создал для тестирования уровня изоляции транзакций. На всех сущностях кроме `Coordinates` я выставил уровень изоляции `SERIALIZABLE`. Так что можно проверить, а действительно ли он нам нужен. Если да, то выставлю и на Coordinates.

Последовательность действий такая:

1. Создаем 2 потока через `ThreadGroup` и авторизуем каждого
2. Синхронизируем через SyncTimer и выпускаем запросы одновременно, создавая одинаковые координаты
3. Делаем запрос на получение всех координат по убыванию id. Если наша затея удалась то мы увидим 2 одинаковых объекта координат в начале. Это нужно проверить jsr-скриптом, например, наслопить на груви.
4. Пробуем обновить одновременни ожин и тот же объект координат. В идеале бы получить либо ошибку, либо не потерять данные
5. Очищаем созданные объекты.

Ожидается, что по итогу параллельного исполнения у нас не будет создано 2 одинаковых объекта координат и мы получим по одному 201 и 409 из-за конфликтного обновления данных.

Самое сложное оказалось написать проверку того, что нет дублей после создания. Я использовал LLM для этого - вот промпт:

> напиши груви скрипт для jsr223 assertion в Apache JMeter.
>
> Предыдущий http eequest возвращает json страницы Spring Data Page в формате:
>
> { «content»: [ { «id»: 1, «x»: 444.25, «y»: 123456789 } ] }
>
> вот что мне нужно:

1. получить тело предыдущего ответа через prev
2. распарсить JSON средствами груви
3. получить ожидаемое значение y из глобального свойства JMeter: props.get(«raceY»).
4. В массиве content найти координаты, у которых:
   - x = 444.25 с допустимой погрешностью 0.0001;
   - y = значению raceY.
5. проверить, что найден ровно один объект
6. Если найдено не ровно один объект, пометить JSR223 Assertion как неуспешный через: AssertionResult.setFailure(true) и дать понятный фидбек в логи через AssertionResult.setFailureMessage()
7. в сообщение об ошибке добавить количество найденных объектов и их id.

верни только готовый Groovy-код с объяснением того что ты использовал.

Вот такой скрипт я получил - он вполне понятный и рабочий.

```
import groovy.json.JsonSlurper

def page = new JsonSlurper().parseText(
    prev.getResponseDataAsString()
)

def expectedY = props.get('raceY') as long

def matches = page.content.findAll { coordinates ->
    Math.abs((coordinates.x as double) - 444.25d) < 0.0001d &&
        (coordinates.y as long) == expectedY
}

log.info(
    "Coordinates uniqueness check: expected x=444.25, y=${expectedY}; "
+
    "found=${matches.size()}, ids=${matches*.id}"
)

if (matches.size() != 1) {
    AssertionResult.setFailure(true)
    AssertionResult.setFailureMessage(
        "Expected exactly one coordinates object, " +
        "but found ${matches.size()}; ids=${matches*.id}"
    )
}
```

Запустив проверку я действительно получил 2 дубля.

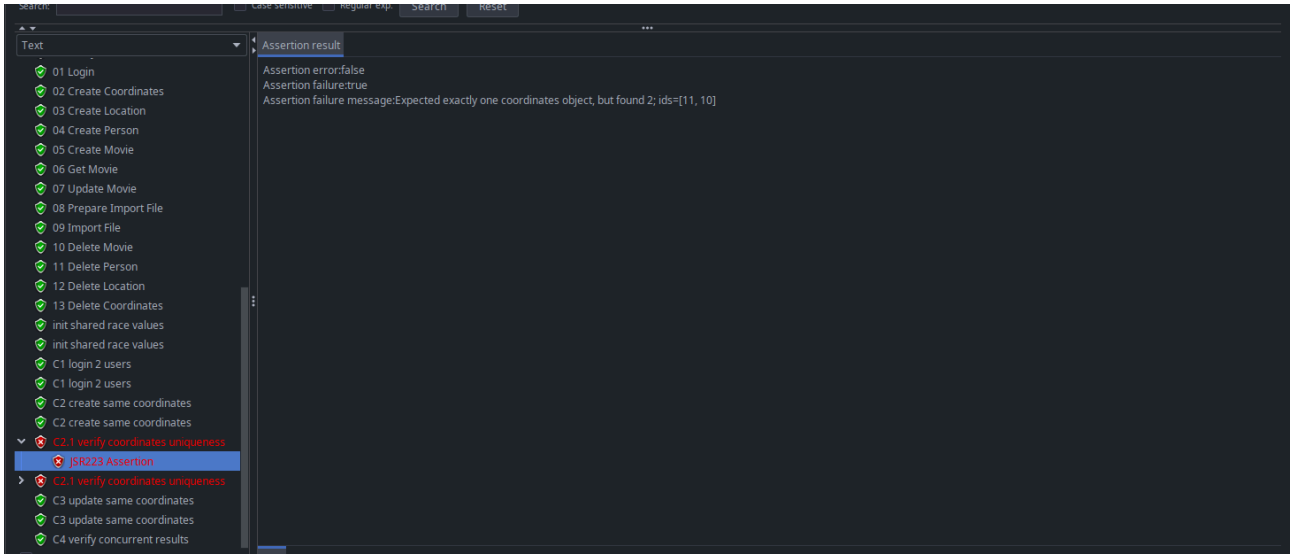



Рис. 4. Неудачное выполнение последовательности действий – данные дублировались

Ну что же – остается навесить `SERIALIZABLE` на координаты и провести тесты заново.

Пришлось также добавить обработку `PessimisticLockingFailureException`, потому что именно ее выбрасывает serializable conflict (у меня просто запятисотило после обновления изоляции)

Сценарий теперь завершился без ошибок.

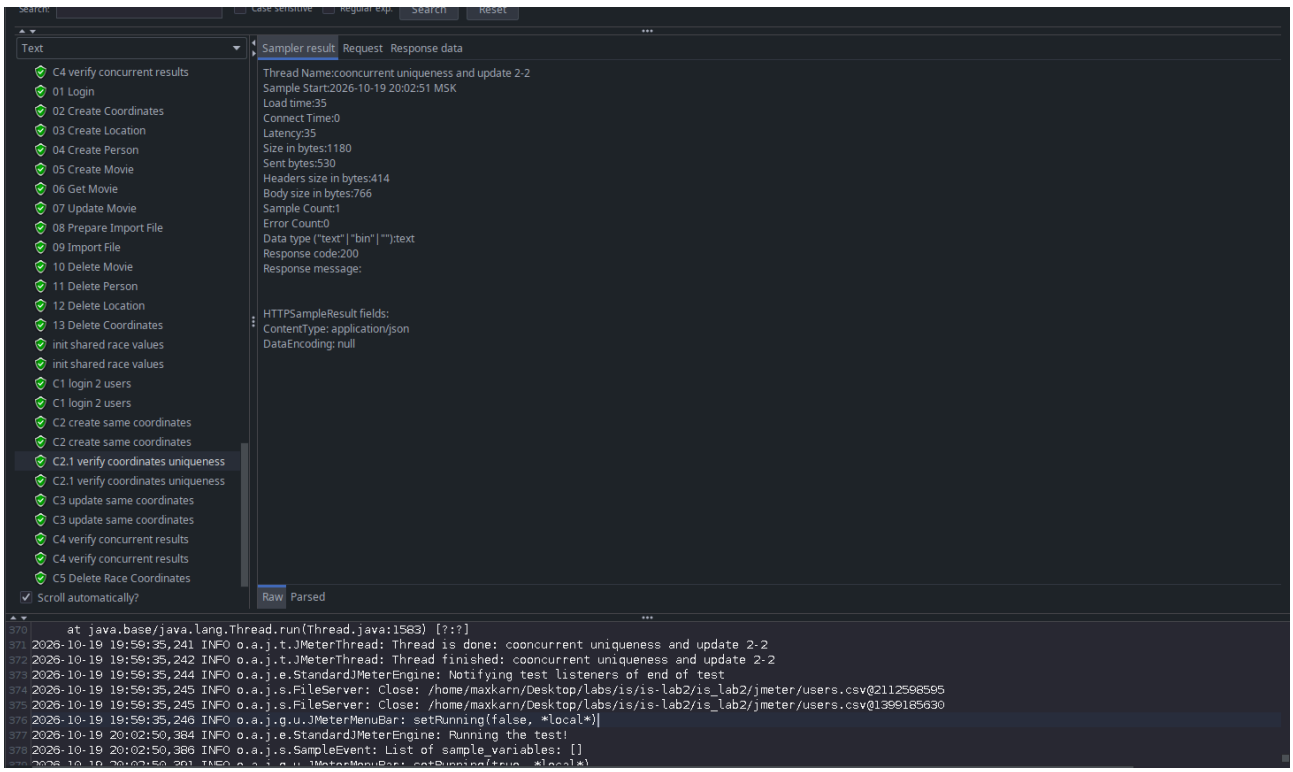



Рис. 5. Успешное выполнение сценария

# 4. Вывод

В ходе выполнения работы я вспомнил транзакции, познкаомился с новым для себя форматом данных hocon - он теперь мой любимый, и второй раз в жизни прикоснулся к Apache JMeter. Провел конкурентное тестирование - это прикольно. Но долго разбирался с тем как вообще программа работает.